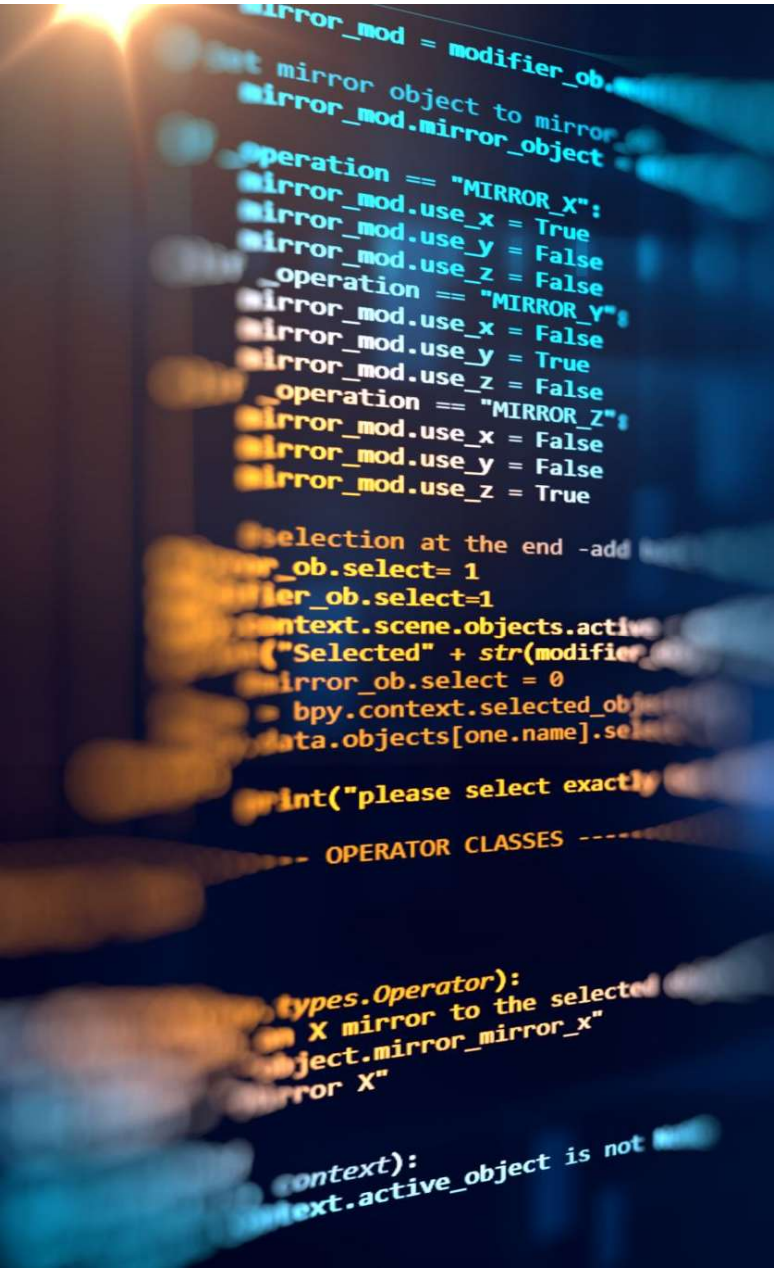
Desenvolvimento Web Front-End

Fabrício Tonetto Londero

Aula 03 – Data Bindings





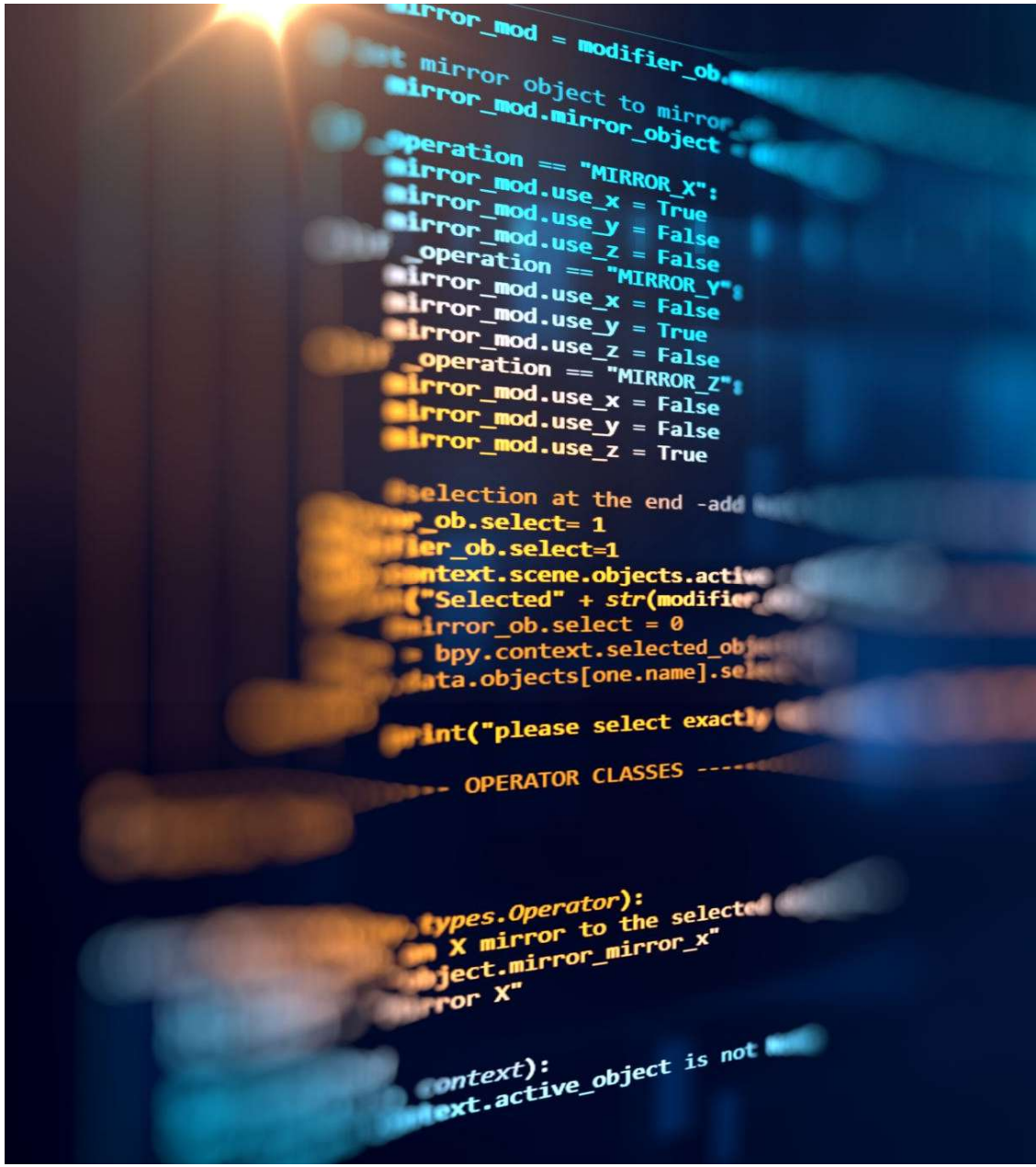
O que é Data Binding?

- Data Binding é o mecanismo do Angular que conecta a lógica do componente (TypeScript) com a interface (HTML).
- Permite que o que está no código apareça na tela e que o que o usuário faz na tela seja enviado para o código.
- É a base da interatividade em aplicações Angular.

Tipos de Data Binding

- Interpolação {{ }}
  - Exibe valores de variáveis do TypeScript no HTML.

Olá, {{ nome }}!



Tipos de Data Binding

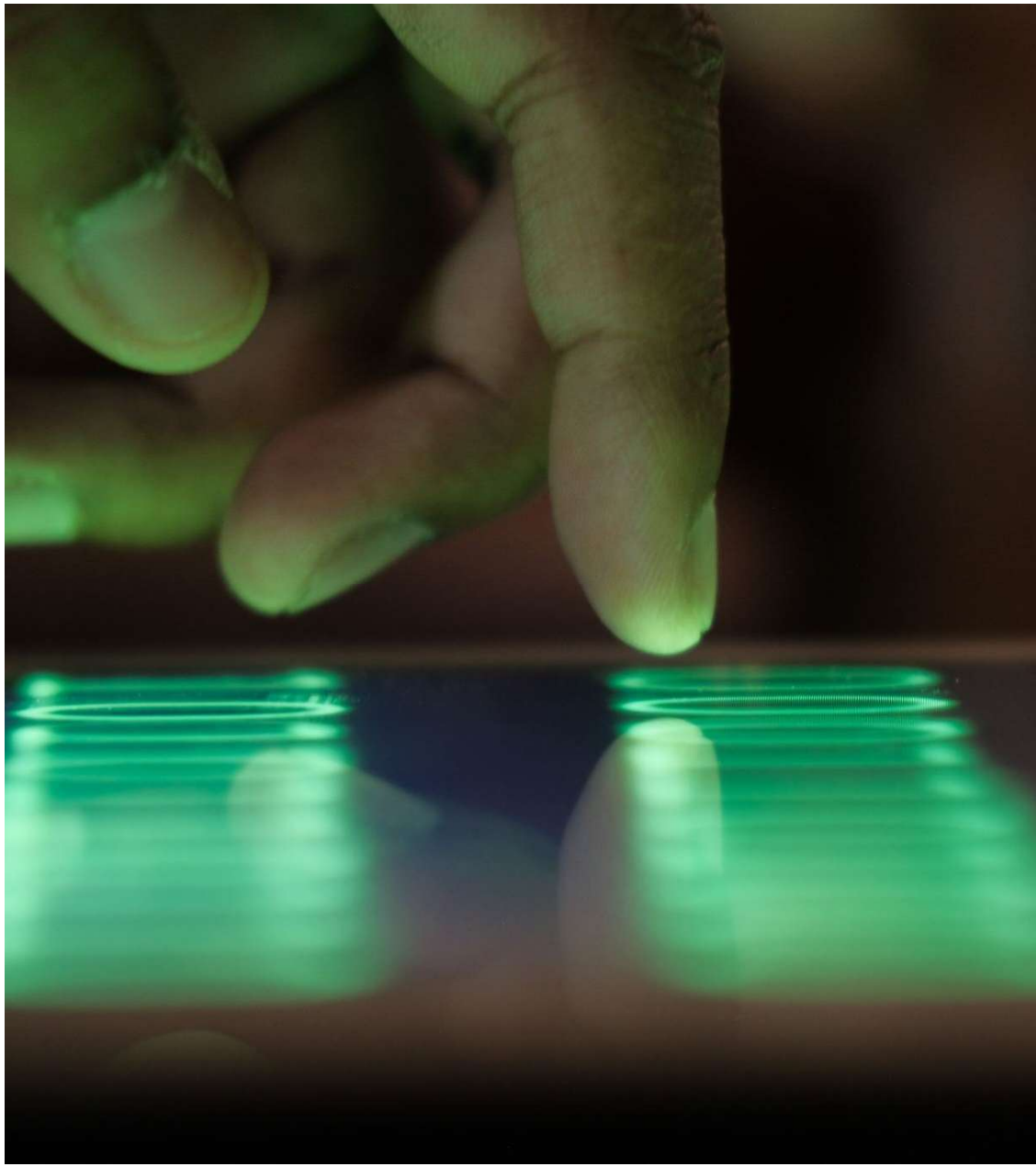
- Property Binding [ ]
  - Liga atributos/propriedades do HTML a valores do TS.

```
<img [src]="urlImagem">
```

Tipos de Data Binding

- Event Binding ( )
  - Captura eventos do usuário (cliques, teclas, etc.) e executa métodos do TS.

<button (click)="enviar()">Enviar</button>



Tipos de Data Binding

- Two-way Data Binding [( )]
  - Conecta o valor de um campo do HTML com o TS em **mão dupla**.

```
<input [(ngModel)]="nome">
```



Por que Data Binding é importante?



Exemplo 1 – HTML e TS do mesmo componente

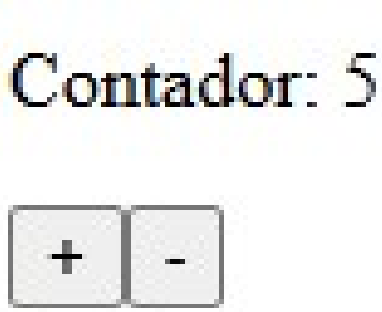
```
<h2>Bem-vindo!</h2>
<p>Usuário: {{ nome }}</p>
```

```
export class NomeDoComponente {
  nome: string = 'Fabrício';
}
```

Exemplo 2

```
<p>Contador: {{ contador }}</p>
<button (click)="incrementar()">+</button>
<button (click)="decrementar()">-</button>
```

```
contador: number = 0;
incrementar() { this.contador++; }
decrementar() { this.contador--; }
```



Exemplo 3

- Para esse exemplo funcionar, precisamos importar a lib FormsModule no nosso TS

```
1  import { Component } from '@angular/core';
2  import { FormsModule } from '@angular/forms';
3
4  @Component({
5    selector: 'app-testes',
6    imports: [FormsModule],
7    templateUrl: './testes.html',
8    styleUrl: './testes.css'
9  })
```

Exemplo 3

```
<input [(ngModel)]="nome" placeholder="Digite seu nome">
<p>Você digitou: {{ nome }}</p>


nome: string = '';
```

Exemplo 4

```
<button (click)="mostrar = !mostrar">Toggle</button>
@if (mostrar) {
  <p>Texto visível!</p>
}
@else {
  <p>Agora está escondido...</p>
}
```

```
mostrar = false;
```

Exemplo 4 – Abordagem clássica

```
<button (click)="mostrar = !mostrar">Toggle</button>
<p *ngIf="mostrar">Texto visível!</p>
```

```
mostrar = false;
//necessário o import
{ CommonModule } from '@angular/common’;
 imports: [CommonModule],
 standalone: true,
```

Resumo

- @if → moderno, não requer import.

- *ngIf → funciona, mas precisa de CommonModule em componentes standalone.

Exemplo 5

```
<ul>
  @for (item of frutas; track $index) {
    <li>{{ item }}</li>
  }
</ul>
```

```
frutas: string[] = ['Maçã', 'Banana', 'Laranja'];
```

Exemplo 6

```
<ul>
  @for (item of frutas; track $index) {
    <li (click)="selecionada = item">{{ item }}</li>
  }
</ul>

@if (selecionada)
{
<p>Você escolheu: {{ selecionada }}</p>
}
```

```
frutas: string[] = ['Maçã', 'Banana', 'Laranja'];
selecionada = '';
```

Exemplo 7

```
<p [style]="{'color': cor}">Texto colorido!</p>
<p [style.color]="cor">Texto colorido!</p>
<input [(ngModel)]="cor"
placeholder="Digite uma cor (ex: red)">
```

```
cor: string = 'blue';
```

Exemplo 8

```html
<p [class]="{'ativo': ativo, 'inativo': !ativo}">
  Estado: {{ ativo ? 'Ativo' : 'Inativo' }}
</p>
<button (click)="ativo = !ativo">Trocar</button>
```

```css
.ativo {
  color: green;
  font-weight: bold;
  border: 1px solid green;
  padding: 5px;
  border-radius: 5px;
}

.inativo {
  color: red;
  font-style: italic;
  border: 1px solid red;
  padding: 5px;
  border-radius: 5px;
}
```

```
ativo = true;
```

Exemplo 9

```
<input (keyup.enter)="adicionarItem($event)"
       placeholder="Digite e pressione Enter">
<ul>
  @for (item of lista; track $index)
  {
    <li>{{ item }}</li>
  }
</ul>
```

```
lista: string[] = [];
 adicionarItem(event: any) {
   this.lista.push(event.target.value);
   event.target.value = '';
 }
```

Exemplo 10

```
<input [(ngModel)]="email" placeholder="Digite seu email">
<button (click)="enviar()">Enviar</button>
@if (!email.includes('@'))
{
  <p>Email inválido</p>
}
```

```
email = '';
  enviar() { alert(`Email enviado para ${this.email}`);
}
```

Exemplo 11

```
<button (click)="like = !like">
  {{ like ? '💖 Curtido' : '♡ Curtir' }}
</button>
```

```
like = false;
```



Exercício 1

- Exiba no template o resultado de um método da classe, por exemplo, getAnoNascimento(), que calcula o ano de nascimento com base na idade.

Exercício 2

- **Controle de estoque**
  - Lista de produtos com nome, quantidade e preço.
  - Exibir o valor total do estoque (quantidade × preço).
  - Permitir adicionar/remover produtos da lista.

Exercício 3

- **Quiz interativo (5 questões)**
  - Mostrar perguntas de múltipla escolha
  - Usuário marca a resposta e o sistema mostra se está certo ou errado.
  - No final, exibir a pontuação.

Exercício 4

- **Gerador de lista dinâmica (CRUD)**
  - Criar, editar e remover itens de uma lista (ex: lista de alunos).
  - Ao editar, o item muda de input para texto.

Exercício 5

- **Controle de acesso (login fake)**
  - Inputs para email e senha.
  - Botão "Login" habilitado apenas se os campos estiverem preenchidos corretamente.
  - Após login, exibir mensagem de boas-vindas.

Exercício 6

- **Carrinho de compras**
  - Lista de produtos com botão "Adicionar ao carrinho".
  - Mostrar o carrinho em tempo real com quantidade e valor total.
  - Possibilidade de remover itens do carrinho.

Exercício 7

- **Tabela com filtro e ordenação**
  - Exibir uma tabela de dados (nome, idade, cidade).
  - Input para filtrar nomes em tempo real.
  - Botões para ordenar por nome ou idade.

Comunicação entre Componentes

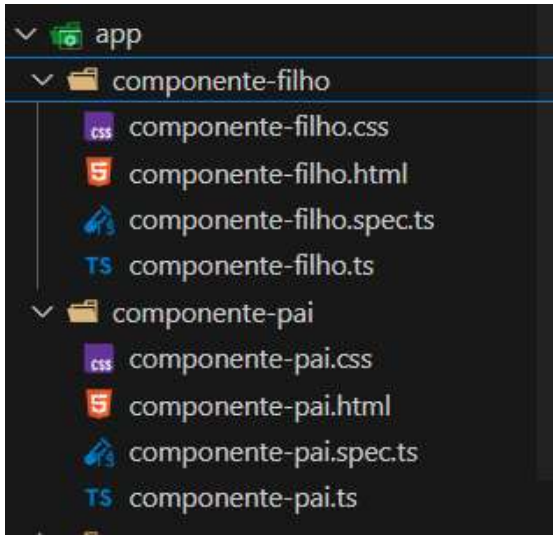
@Input() — permite que o pai passe dados para o filho via property binding ([algumaCoisa]="valorDoPai").

@Output() — permite que o filho envie eventos para o pai, usando um EventEmitter ((meuEvento)="handlerDoPai($event)").

Juntos eles permitem fluxo de dados em “baixo acoplamento”: pai controla dados; filho notifica o pai sobre ações.

Comunicação entre Componentes

- ng g c componentePai
- ng g c componenteFilho



Exemplo 1 – Componente Filho

```
export class ComponenteFilho
{
  @Input() message!: string;
}
```

```
<p>Mensagem do pai: {{ message }}</p>
```

Exemplo 1 – Componente Pai

```ts
import { Component } from '@angular/core';
import { ComponenteFilho } from "../componente-
filho/componente-filho";

@Component({
  selector: 'app-componente-pai',
  imports: [ComponenteFilho],
  standalone: true,
  templateUrl: './componente-pai.html',
  styleUrl: './componente-pai.css'
})
export class ComponentePai {
 parentMessage = 'Olá, filho!';
}
```

```html
<h2>Componente Pai</h2>
<app-componente-filho
[message]="parentMessage"></app-componente-
filho>
```

Exemplo 1


Home | Login | Contato | Sobre | Test
Componente Pai
Mensagem do pai: Olá, filho!

Exemplo 2 – Componente Filho

```
@Output() messageEvent = new EventEmitter<string>();

  sendMessage()
  {
    this.messageEvent.emit('Olá, pai! Mensagem do filho.');
  }
```

```
<button (click)="sendMessage()">Enviar mensagem ao pai</button>
```

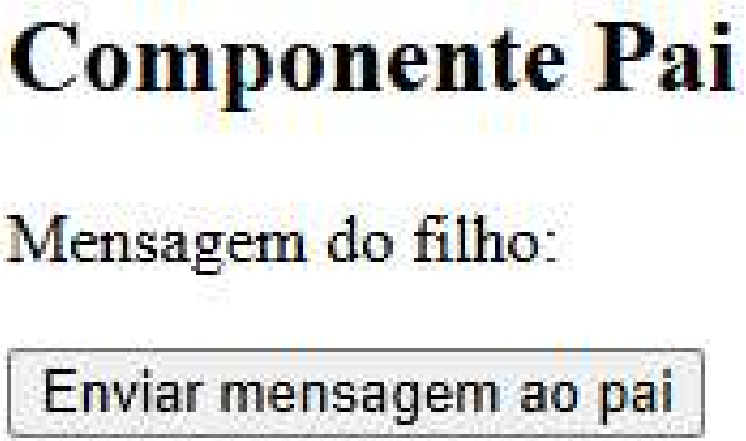
Exemplo 2 – Componente Pai

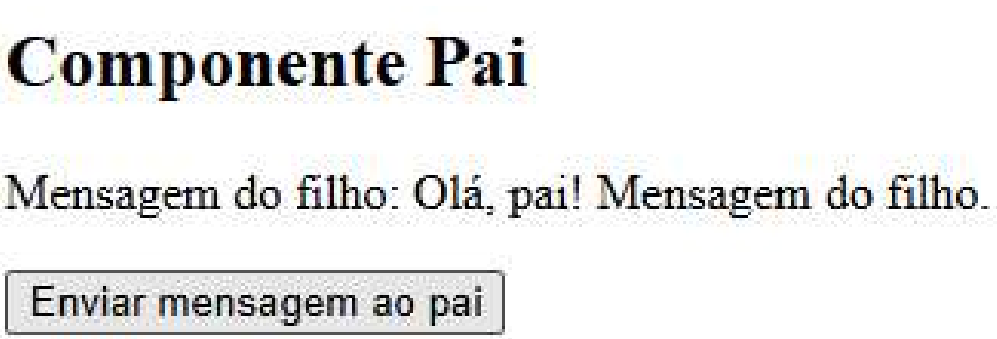
```
childMessage = '';

  receiveMessage(msg: string) {
    this.childMessage = msg;
  }
```

```
<h2>Componente Pai</h2>
<p>Mensagem do filho: {{ childMessage }}</p>
<app-componente-filho (messageEvent)="receiveMessage($event)"></app-componente-filho>
```

Exemplo 2


Componente Pai
Mensagem do filho:
Enviar mensagem ao pai


Componente Pai
Mensagem do filho: Olá, pai! Mensagem do filho.
Enviar mensagem ao pai

Exercício 1

- Lista de Tarefas (Pai envia → Filho exibe)
  - O componente Pai tem uma lista de tarefas (['Estudar Angular', 'Fazer exercícios', 'Ler livro’]).
  - O componente Filho recebe a lista via @Input() e exibe

Exercício 2

- Carrinho de Compras
  - O componente Pai tem uma lista de produtos disponíveis.
  - Cada componente Filho representa um produto, com botão "Adicionar ao carrinho".
  - O Pai recebe o evento via @Output e atualiza o carrinho.
  - O Pai também envia informações para os filhos via @Input (por exemplo, se o produto já está no carrinho).